Problem A. Nezha Naohai

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	64 megabytes

Nezha caused conflicts due to the dominance of the Dragon Palace. In order to uphold justice and protect the people, Nezha caused three incidents in the sea. The specific process and outcome are as follows:

The 1st sea-tussle: Slayed the Yaksha Li Gen, took A hours;

The 2nd sea-tussle: Subdued Ao Bing, the third son of the Dragon King, took B hours;

The 3rd sea-tussle: Forced the Dragon King to submit, took C hours.

Nezha's disturbance in the sea would cause complaints. For every hour he causes trouble in the sea, he would generate D complaints. So, how many complaints did Nezha's disturbance in the sea cause in total?

Input

There are four integers A,B,C and D ($1 \le A, B, C, D \le 100$) respectively represent the time spent by Nezha in the sea for three times and the number of complaints caused by the sea disturbance per hour.

Output

The total number of complaints caused by Nezha's three disturbances in the sea.

Example

standard input	standard output
1 2 3 4	24

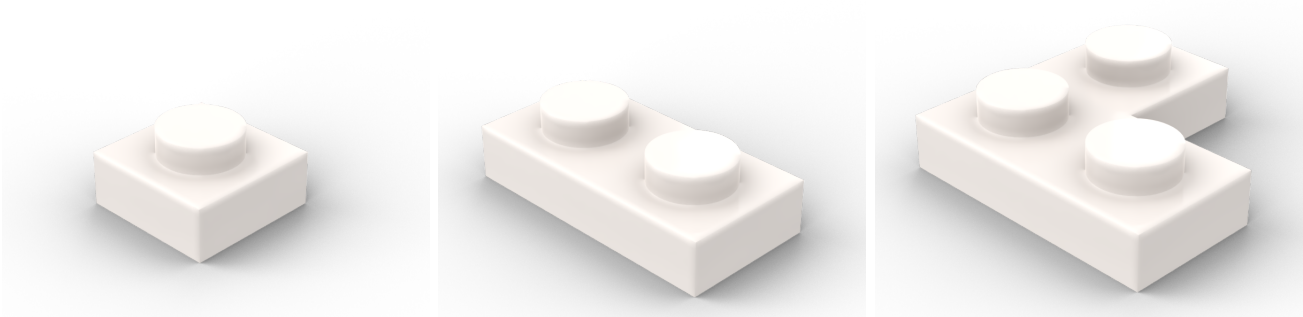
Problem B. LEGO-complete

Input file:	`standard input`
Output file:	`standard output`
Time limit:	5 seconds
Memory limit:	512 megabytes

You are given a binary matrix consisting of 0s and 1s. All the 1s form a single connected component under 4-directional connectivity (i.e., cells are connected if they share a side: up, down, left, or right).

Your task is to build a 2-layer LEGO structure exactly covering the region of 1s using the following three types of bricks:

- **1×1 brick**: covers exactly one cell.
- **1×2 brick**: covers two adjacent cells, and can be placed either horizontally or vertically.
- **L-shape brick**: covers three cells forming an "L"shape — for example, a cell and its top and right neighbors. All four 90-degree rotations of this shape are allowed.



Your goal is to cover all the 1s using the specified types of bricks on **both two layers** while leaving all the 0s empty on **both two layers**. In addition, the entire LEGO structure must be structurally stable — that is, it **Does Not Fall Apart**.

We define two bricks to be **adjacent** if they are in different layer and share at least one common cell. A LEGO structure is said **Does Not Fall Apart** if all bricks form a single connected component through such adjacency.

If any solution exists, output any one of them. Otherwise, output -1.

Formally, you are given an $n \times m$ binary matrix A. You need to output two $n \times m$ matrices B_0 and B_1 representing the brick IDs on the upper and lower layers. Your output must satisfy the following:

- **Exact Coverage**: If $A_{x,y} = 1$, then $B_{0,x,y}, B_{1,x,y} \in [1, 2nm]$. If $A_{x,y} = 0$, then $B_{0,x,y} = B_{1,x,y} = 0$.
- **Valid Bricks**: Each brick ID appears only on one layer. Its occupied cells must form a valid shape: 1×1, 1×2, or L-shaped.
- **Does Not Fall Apart**: Define two bricks i and j **adjacent** if there exists (x, y) such that $\{B_{0,x,y}, B_{1,x,y}\} = \{i, j\}$. All bricks occur in B must form a single connected component through such adjacency.

Input

The first line contains two integers n, m ($1 \le n, m \le 1 \times 10^3$).

For the following n lines, each line contains a 01 string with length m. Represents matrix A. It is guaranteed that all the 1s in A under 4-directional connectivity.

Output

If any solution exists, output the two matrices B_0 and B_1 as follows: for each matrix, print n lines, each line containing m integers separated by spaces.

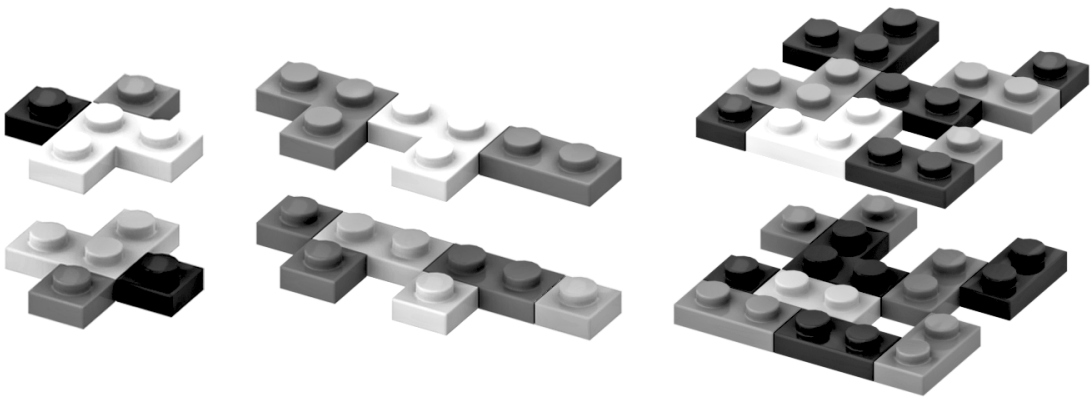
If no such solution exists, output -1.

Examples

standard input	standard output
3 3 010 111 010	0 3 0 4 1 1 0 1 0 0 2 0 2 2 5 0 6 0
2 6 111111 010100	1 1 2 2 5 5 0 1 0 2 0 0 6 3 3 4 4 7 0 8 0 9 0 0
5 5 01001 11011 01110 11101 11111	0 1 0 0 15 1 1 0 6 6 0 3 8 8 0 3 3 4 0 18 19 4 4 13 13 0 14 0 0 5 16 2 0 7 5 0 2 2 7 0 17 9 9 0 10 11 11 12 12 10

Note

The picture shows how examples are constructed.



Problem C. Osiris

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	512 megabytes

"My name is D'Arby, D-A-R-B-Y. The "D" has an apostrophe."

The Stardust Crusaders, after a long journey, finally arrived at their destination—Cairo, Egypt. They immediately visited a nearby café to inquire about Dio's mansion. A customer there calmly remarked, "I recall that mansion." When Joseph Joestar pressed for details, the customer demanded a gamble in exchange for information. Joseph and Polnareff wagered against this customer, the enemy Stand Master Daniel J. D'Arby, but lost and had their souls stolen. To reclaim their souls, Kujo Jotaro challenges D'Arby...

Jotaro and D'Arby play a poker game under these rules: They use a shuffled 52-card deck (no jokers)*, and each initially draws 5 random cards. Players know only their own hands. Jotaro can discard k ($k > 0$) of his cards (*without returning them to the deck*) and draw k new cards from the remaining deck. After this, both reveal their hands and compare the total point sums. If Jotaro's sum exceeds D'Arby's, he gains k chips; if his sum is smaller, he loses k chips; if equal, the chips remain unchanged.

Jotaro wants to compute the maximum expected value of chips he can obtain for each $k = 1, 2, \cdots, 5$, assuming he makes optimal decisions when discarding and redrawing cards. Specifically, for each k, calculate the expected value of the maximum possible chip outcome.

All answers must be computed modulo 998244353†.

Input

One line with five space-separated values (integers or uppercase letters) representing Jotaro's initial cards.

Valid inputs are: `A`, `2-10`, `J`, `Q`, `K`.

It is guaranteed that each value in the input will appear at most 4 times, as there are only 4 cards of each value in the deck.

Output

Five lines, where the i-th line ($1 \le i \le 5$) contains the expected value for $k = i$, modulo 998244353.

Examples

standard input	standard output

`A A A A Q`	`79189732`
	`281328086`
	`60649431`
	`607386498`
	`0`

`8 6 8 10 J`	`672098766`
	`396700559`
	`282487893`
	`144365991`
	`0`

*The deck contains 52 cards with 13 ranks (`A`, `2-10`, `J`, `Q`, `K`) and 4 suits per rank. Here, `A` is treated as 1, `J` as 11, `Q` as 12, and `K` as 13.

†Regarding modulo operation on fractions: For a fraction $\frac{a}{b}$, the calculation formula is $\frac{a}{b} \bmod p = (a \cdot b^{-1}) \bmod p$, where b^{-1} is the multiplicative inverse of b under modulo p, satisfying $b \cdot b^{-1} \equiv 1 \pmod p$. For example, under modulo 5, the inverse of 3 is 2, because $3 \times 2 \equiv 1 \pmod 5$. Therefore, $\frac{2}{3} \bmod 5 = (2 \times 2) \bmod 5 = 4$.

Problem D. Virtuous Pope

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	512 megabytes

"Take this, DIO! 20-meter radius Emerald Splash—!"

The Stardust Crusaders, after overcoming countless obstacles and repelling waves of enemies, finally arrived in Egypt for their ultimate confrontation with Dio. Facing Dio, their strategy involved Joseph Joestar and Kakyoin engaging in tactical retreats while fighting, while Kujo Jotaro and Polnareff pursued from the rear, forming a pincer attack. Amidst the chaos, Kakyoin devised a plan to uncover the abilities of Dio's Stand "The World", by setting traps across nearby structures.

As Dio pursued Kakyoin, he became ensnared in the barrier deployed by Kakyoin's Hierophant Green. The ensuing confrontation unleashed a concentrated barrage of Emerald Splash attacks...

Kakyoin has deployed a barrier of "Hierophant Green" within a cuboid region. For calculation purposes, we place the cuboid in a Cartesian coordinate system, with one vertex at the origin $(0, 0, 0)$ and the opposite vertex at (a, b, c), where a, b, and c represent the length, width, and height of the cuboid respectively. The barrier contains several tentacles, each defined as a line segment from (x_1, y_1, z_1) to (x_2, y_2, z_2), where both endpoints lie on the boundary of the cuboid.

Dio may appear at any position within the cuboid (including boundaries). According to the plan, Dio will execute a single attack by generating an infinitely large plane perpendicular to one of the coordinate axes ($x-$, $y-$, or $z-$ axis) and centered at his position. All tentacles intersecting Dio's attack range (i.e., there exists a point that lies on both the tentacle and Dio's attack plane) will be severed. Each severed tentacle will launch an Emerald Splash attack against Dio exactly once.

Kakyoin wants to determine: What is the maximum number of Emerald Splash attacks Dio could receive considering all possible positions he might occupy and all valid attack directions after the barrier is deployed?

Formalized statement: Given n line segments in three-dimensional space with their endpoints restricted to a given cuboid, determine the maximum number of segments intersected by any plane that is perpendicular to one of the coordinate axes. A line segment is considered to intersect a plane if and only if there exists a point that lies on both the line segment and the plane.

Input

The first line contains four integers n, a, b, c $(1 \le n \le 10^5, 1 \le a, b, c \le 10^9)$:

- n: number of "Hierophant Green" 's tentacles;
- a, b, c: dimensions of the cuboid (length, width, height).

The next n lines each contain six integers $x_1, y_1, z_1, x_2, y_2, z_2$ $(0 \le x_1, x_2 \le a, 0 \le y_1, y_2 \le b, 0 \le z_1, z_2 \le c)$, describing coordinates of the two endpoints (x_1, y_1, z_1) and (x_2, y_2, z_2) for each tentacle. All endpoints lie on the cuboid's surface.

Output

Output a single integer: the maximum number of Emerald Splash attacks Dio could receive.

Examples

standard input	standard output
3 2 2 2 1 1 0 1 1 2 1 0 1 1 2 1 0 1 1 2 1 1	3
4 10 4 10 0 4 6 6 4 4 10 2 10 8 4 4 0 4 0 2 4 0 0 0 10 0 2 10	3

Note

For Sample 1: When Dio's attack plane is set to $x = 1$, he will receive 3 Emerald Splash attacks.

For Sample 2: When Dio's attack plane is $x = 0$, he will receive 3 attacks.

Problem E. God's String on This Wonderful World

Input file:	`standard input`
Output file:	`standard output`
Time limit:	3 seconds
Memory limit:	1024 megabytes

We use $|s|$ to denote the length of string s, and s_i to denote the i-th character of s. A substring of s, denoted $s_{l...r}$ $(1 \le l \le r \le |s|)$, refers to the string formed by concatenating the $r - l + 1$ characters $s_l, s_{l+1}, \ldots, s_r$. We say that s is a k-fold string if and only if there exists a string t such that $s = t^k$, i.e., k copies of t concatenated end-to-end exactly form s.

Aqua wants to know, given string s and positive integers x, y, how many pairs of positive integers l, r satisfy $x \le l \le r \le y$ and the substring $s_{l...r}$ can be rearranged to become a k-fold string. Since Aqua likes high-performance programs, you need to answer q queries $(x_1, y_1), (x_2, y_2), \ldots, (x_q, y_q)$.

Input

The first line contains three positive integers n, k, q $(1 \le n, k, q \le 3 \times 10^5)$, representing the length of the string, the fold count for queries, and the number of queries.

The second line contains a string s of length n. It is guaranteed that s consists only of lowercase English letters.

The following q lines each contain two positive integers x, y $(1 \le x \le y \le n)$, representing one query per line.

Output

Output q lines, where the i-th line contains an integer representing the answer to the i-th query.

Example

standard input	standard output
14 2 6 ssessesessefpq 1 5 1 6 3 6 10 14 1 14 2 7	2 4 2 0 12 3

Note

For the first query, the pairs (l, r) that satisfy the condition are $(1, 2), (4, 5)$.

For the second query, the pairs (l, r) that satisfy the condition are $(1, 2), (1, 6), (3, 6), (4, 5)$.

Problem F. Caloric Difference

Input file:	`standard input`
Output file:	`standard output`
Time limit:	2 seconds
Memory limit:	256 megabytes

Red Dragon Dew's master believes that she has been eating too much recently, causing her to wobble when taking off and reducing her cruising time. Therefore, he ordered her to control her caloric intake.

Over the next n days, suppose that on the i-th day, Dew consumed r_i calories and metabolized c_i grams of mass. According to the characteristics of her dragon species, the mass loss should be $c_i = p \times c_{i-1} + (1-p) \times r_{i-1}$, where the parameters r_0, c_0, and p $(0 < p < 1)$ have been measured by her master.

Of course, Red Dragon Dew indicated that she would not fully obey commands from her "cook" . She stated that over the next n days, there are k days when she will have indulgent meals. The caloric intake on these k days is predetermined, and the daily caloric intake must be within the range $[L, R]$, i.e. all r_i should in range $[L, R]$.

Given these conditions, Dew wants to arrange the days without indulgent meals to maximize the difference between the total caloric consumption and intake over the next n days, i.e. $\max \sum_{i=1}^{n}(c_i - r_i)$.

Input

Each test case consists of multiple test cases. The first line contains the number of test cases T $(1 \le T \le 10000)$. The description of the test cases follows:

The first line contains two integers n and k $(1 \le k \le n \le 10^5,\ 1 \le \sum n \le 2 \times 10^5)$, indicating that there are n days ahead, and k days have predetermined caloric intake.

The second line contains five real numbers r_0, c_0, p, L, and R $(1 \le L \le r_0 \le R \le 10^9,\ 0 < p < 1,\ 1 \le c_0 \le 10^9)$, representing the parameters mentioned above.

The next k lines, each containing an integer p_i $(1 \le p_i \le n,\ \forall i \ne j, p_i \ne p_j)$ and a real number v_i $(L \le v_i \le R)$, indicating that r_{p_i} is fixed at v_i.

Ensure that all input real numbers have at most 6 decimal places.

Output

For each test case, output the maximum value of $\sum_{i=1}^{n}(c_i - r_i)$.

Your answer is considered correct if its absolute or relative difference to the correct answer is at most 10^{-6}.

More formally, let a be your answer and b be the correct answer. Your output is considered correct if $\frac{|a-b|}{\max(1,b)} \le 10^{-6}$.

Example

standard input	standard output
2 3 2 5 6 0.5 1 10 1 4 2 5 5 2 10 5 0.5 3 12 1 4 3 6	5.1250000000 7.9062500000

Note

For the first testcase, one optimal possible $r = [5, 4, 5, 1]$ and its corresponding $c = [6, 5.5, 4.75, 4.875]$.

Problem G. Exploration

Input file:	`standard input`
Output file:	`standard output`
Time limit:	2 seconds
Memory limit:	1024 megabytes

Alice is exploring a dangerous large maze. This maze has n caves (labeled from 1 to n) and m one-way passages connecting any two of these caves (because some traps in the passages make it difficult to return once passed). Each passage is represented as (u, v), where u is the starting cave and v is the destination cave. Each passage (u, v) has a difficulty value $d_{(u,v)}$. In other words, you can think of the entire cave system as a weighted directed graph.

At the beginning, Alice has an initial stamina (or it can be called 'energy') x. When she traverses a passage with difficulty value d, her stamina x changes to $\lfloor \frac{x}{d} \rfloor$ (where $\lfloor\ \rfloor$ is the floor function). When Alice's stamina drops to 0, it is considered that her stamina is exhausted, and she can no longer continue exploring.

For a example: Suppose Alice starts with a stamina value of 9. After traversing a passage with difficulty 2, her stamina becomes 4 (since $\lfloor \frac{9}{2} \rfloor = 4$). If she then traverses another passage with difficulty 2, her stamina drops to 2 ($\lfloor \frac{4}{2} \rfloor = 2$). Finally, after traversing a passage with difficulty 3, her stamina becomes 0 ($\lfloor \frac{2}{3} \rfloor = 0$), at which point her stamina is exhausted.

Since Alice has a poor sense of direction in the maze, at any point during her exploration, she does not know which cave she is currently in or where each passage leads. Therefore, Alice has Q questions. For the i-th question, she wants to ask: If she starts from cave p_i with an initial stamina of x_i, and each time she randomly chooses a passage she can take, what is the minimum number of passages she must traverse before her stamina is exhausted? (**Note**: If Alice traverses a passage she has taken before, her stamina will still decrease in the same way, i.e., from x to $\lfloor \frac{x}{d} \rfloor$.)

It is guaranteed that every cave has at least one outgoing one-way passage (the starting and ending caves of a passage may be the same).

Input

The first line contains three integers n, m, Q ($1 \le n, Q \le 2 \times 10^5, n \le m \le 5 \times 10^5$), representing the number of caves, the number of passages, and the number of queries respectively.

The following m lines each contain three integers u, v, d ($1 \le u, v \le n, 2 \le d \le 10^9$), indicating a one-way passage from u to v with difficulty value d.

It is guaranteed that $\forall i \in [1, n], \exists u, \text{s.t. } u = i$.

The next Q lines each contain two integers p_i, x_i ($1 \le p_i \le n, 1 \le x_i \le 10^9$),representing the i-th query: if Alice starts from cave p_i with initial stamina x_i, what is the minimum number of passages she must traverse before her stamina is exhausted.

Output

Output Q lines, each containing a single integer A_i, indicating that for the i-th query, Alice will exhaust her stamina after traversing a minimum of A_i passages.

Example

standard input	standard output
3 4 7 1 2 2 2 3 4 3 2 3 3 3 2 1 10 2 9 3 2 2 8 3 1 1 7 2 4	3 2 1 2 1 2 2

Problem H. Bingo Game

Input file:	`standard input`
Output file:	`standard output`
Time limit:	5 seconds
Memory limit:	256 megabytes

Red Dragon Dew forwarded a picture of a Bingo game:

The above is an example of a 5×5 Bingo game.

For an $n \times n$ Bingo game, the player must read the description on each cell and check off all the cells that satisfy the description. If the checked cells form at least one line (**horizontally, vertically, or along either of the two diagonals**), then the player is considered to have succeeded in the game.

Now, Red Dragon Dew wants to know the maximum number of cells that can be checked while ensuring the player still fails. That task is too simple, so she further wants to know: how many different schemes are there for checking the maximum number of cells such that the player fails?

Specifically, if there is a cell that is unchecked in one scheme and checked in another, then the two schemes

are considered different. To avoid excessively large outputs, the final answer is taken modulo 998244353.

Input

Each test contains multiple test cases. The first line contains the number of test cases T ($1 \le T \le 10$). The description of the test cases follows:

The only line of each test case contains an integer n ($2 \le n \le 10^5$), which represents the size of the Bingo game.

Output

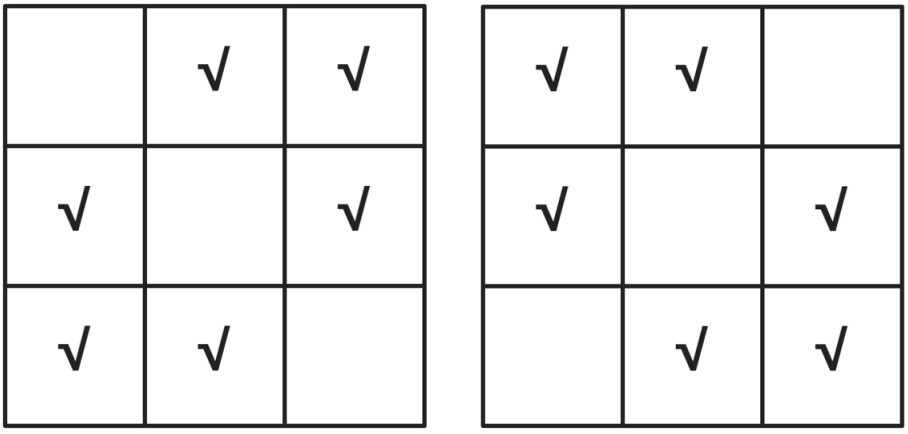
For each test case, output the number of schemes for checking the maximum number of cells that (still result in failure), modulo 998244353.

Example

standard input	standard output
4 3 7 114 997	2 2004 293058554 136105176

Note

Here is 3×3 bingo game schemes for checking the maximum number of cells which still results in failure.



Problem I. Dating Day

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	512 megabytes

TreeQwQ is busy dating with girls. There are n time slots in a day numbered from 1 to n. TreeQwQ is going on some dates today. He will give you a binary string s to describe his schedule. $s_i = 1$ denotes TreeQwQ is dating in time slot i while $s_i = 0$ denotes he is free in time slot i.

However, he finds his schedule boring because he has too many girls to date with. So you can help him by performing the following operation on his schedule **exactly once**.

- First, select an range $[l, r](1 \le l \le r \le n)$ such that there are exactly k dating time slots from time slot l to time slot r(inclusive).
- After selection, you can arbitrarily modify the states of the time slots(that is changing a time slot from dating to free, vice versa) from the l-th to the r-th time slot any times.
- After all the modifications, you need to guarantee that there are still exactly k dating time slots from time slot l to time slot r(inclusive).

TreeQwQ wants to know how many possible schedules he can get after performing the operation. Since the number may be very large. You need to tell TreeQwQ the number modulo 998244353.

Note that two schedules are called different if there exists a time slot that TreeQwQ is dating in one schedule while he is free in the same time slot in the other schedule.

Input

The first line of the input contains a single integer t $(1 \le t \le 10^5)$ indicating the number of test cases.

The first line of each test case consists of two integers n, k $(1 \le n \le 10^5, 1 \le k \le n)$ indicating the number of time slots and the required number of dating time slots in the chosen interval.

The second line of each test case contains a binary string of length n, describing TreeQwQ's schedule.

It is guaranteed that the sum of n over all test cases does not exceed 10^6.

Output

For each test case, output one integer, the number of possible scheduels TreeQwQ can get modulo 998244353.

Example

standard input	standard output
3 6 2 101011 7 4 1100111 4 2 0010	10 20 0

Problem J. Hot Pepper

Input file:	`standard input`
Output file:	`standard output`
Time limit:	2 seconds
Memory limit:	256 megabytes

Red Dragon Dew has recently become obsessed with Plants vs. Zombies Fusion Edition (by the Lan Piaopiao team). Like the original Plants vs. Zombies, there is a plant called the Hot Pepper, which can damage all zombies in a row simultaneously. Based on this, Dew came up with a problem:

On an infinite two-dimensional grid, each cell can hold **at most one pepper**. There are two types of peppers: one that explodes **vertically** (parallel to the y-axis) and one that explodes **horizontally** (parallel to the x-axis). Now, there are already k peppers with fixed types on the grid, and Dew wants to know the minimum number of additional peppers that need to be placed so that every pepper is burned by at least one other pepper.

Formally, given a set of triples $S = \{(x, y, w)\}$, where $(x, y, w) \in U$ and $U = N \times N \times \{0, 1\}$, the problem asks for the minimum number of additional elements $(x, y, w) \in U$ to be added to S such that for every $(x_0, y_0, w_0) \in S$, there exists either an element $(x_0, y', 0) \in S, y' \neq y_0$ or an element $(x', y_0, 1) \in S, x' \neq x_0$. Meanwhile, the set must satisfy that for any two elements (x_1, y_1, w_1) and (x_2, y_2, w_2) in S, it is not the case that both $x_1 = x_2$ and $y_1 = y_2$ simultaneously.

Input

Each test contains multiple test cases. The first line contains the number of test cases T ($1 \le T \le 50000$). The description of the test cases follows. The first line of each test case contains an integer k ($1 \le k \le 2 \times 10^5$), which represents the number of peppers.

The next k lines, each line contains 3 integers x, y, w ($1 \le x, y \le 10^9, w \in \{0, 1\}$), representing the position and type of each pepper, with the guarantee that no two peppers share the same position.

For all test cases, it is guaranteed that $\sum k \le 2 \times 10^5$.

Output

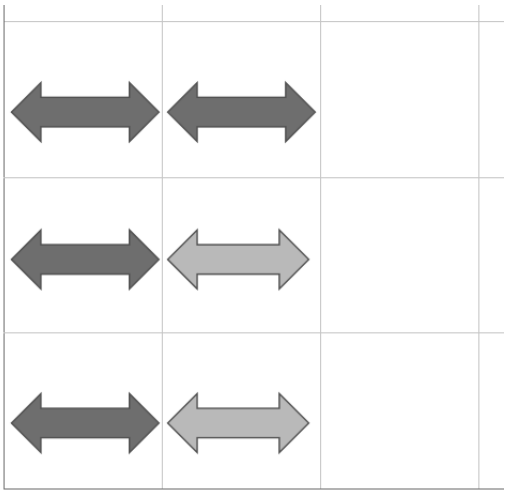
For each test case, output the minimum number of additional peppers required.

Examples

standard input	standard output
3 4 1 1 1 1 2 1 1 3 1 2 3 1 3 1 1 0 2 1 0 2 2 1 6 1 3 0 2 3 1 2 2 0 3 2 1 3 1 0 1 1 1	2 2 0
3 3 1 1 0 2 1 0 3 2 1 4 1 1 0 2 1 0 3 2 1 3 3 1 2 1 1 0 2 2 1	3 3 2

Note

Here is the first test case for example 1, assume the lower left corner is (1,1), dark arrows are the peppers original, and two light arrows are one possible minimum peppers added scheme.



Problem K. Rotation

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

You have n stone statues, each with a nose. The statues can face 4 directions: front, right, back, or left.

Each time you can either:

- Press the nose of one statue, causing all other statues to rotate 90° clockwise simultaneously; or
- Press your own nose, causing all statues to rotate 90° clockwise.

The clockwise rotation sequence is: front → right → back → left.

What is the **minimum number of presses** needed to make all statues face front?

Input

The first line contains an integer n ($1 \le n \le 10^6$).

The next line contains n space-separated integers a_i ($a_i \in \{0, 1, 2, 3\}$) representing directions, $a_i = 0, 1, 2, 3$ correspond to **front, right, back, left** respectively.

Output

A single integer representing the answer.

Example

standard input	standard output
4 0 1 2 3	6

Problem L. Regnaissance

Input file:	`standard input`
Output file:	`standard output`
Time limit:	4 seconds
Memory limit:	1024 megabytes

redemption

nothingness

rebirth

Why are you crying?

Luvia gives you a tree with n nodes numbered $1 \sim n$, and q queries l, r, x,

you should answer the LCA of nodes numbered $l \sim r$ when the root of the tree is x.

LCA means the lowerest common ancestor.

Input

The first line, two integers n, q $(1 \le n, q \le 3 \times 10^5)$.

Each of the next $n-1$ lines contains two integers x, y $(1 \le x, y \le n)$, indicating an edge from node x to node y.

It is guaranteed that the given edges form a tree.

The next q lines, each contains a query with three integers l, r, x $(1 \le l \le r \le n, 1 \le x \le n)$.

Output

Output q lines, line i contains one integer ——the answer of the i-query.

Example

standard input	standard output
8 6 1 2 1 3 1 7 2 4 3 5 3 6 5 8 5 6 1 1 4 7 6 8 4 1 8 4 4 7 2 5 5 3	3 1 1 4 2 5

Problem M. Divide coins

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

Little T and Little J are playing a coin-flipping game with n coins, all initially facing up. First, they agree on a number k. Little T then flips exactly k coins to face down, but Little J does not know which ones.

After that, Little J must assign each coin to one of four operations:

1. Place in the first pile (no flip)
2. Place in the first pile and flip
3. Place in the second pile (no flip)
4. Place in the second pile and flip

Example: When $n = 5$ with operation sequence `12341`:

- Coins 1, 2, 5 $\rightarrow$ first pile (coin 2 is flipped)
- Coins 3, 4 $\rightarrow$ second pile (coin 4 is flipped)

The game is won by Little J if and only if **both piles have the same number of coins facing up** after all operations. **One of the piles is allowed to contain zero coins.**

Your task is to determine whether there exists an operation sequence that guarantees Little J's victory **for any possible choice of k flipped coins** by Little T. If such a sequence exists, construct it; otherwise, output -1.

Input

Integers n and k ($1 \le n \le 10^4, 0 \le k \le n$).

Output

A valid operation string of length n (using characters $1 - 4$), or -1 if impossible

Example

standard input	standard output
4 4	1234